

ROYAL COLLEGE OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CST 306: ALGORITHM ANALYSIS & DESIGN

MODULE I

1. Is $n^2 \in o(n^2)$? (Little o). Justify your answer.
2. Let $f(n) = 7n + 4$. Prove that this is of the order of $\Omega(n)$.
3. Illustrate best case, average case and worst-case complexity with insertion sort algorithm.
4. Define asymptotic notation? Arrange the following functions in increasing order of asymptotic growth rate.

$$n^3, 2^n, \log n^3, 2^{100}, n^2 \log n, n^n, \log n, n^{0.3}, 2^{\log n}$$

5. Is $2^{n+1} = O(2^n)$?
6. Is $2^{2n} = O(2^n)$?
7. Show that $\log n! = \Theta(n \log n)$
8. Let $f(n) = 3n^3 + 2n^2 + 3$ for an algorithm, Let $g(n) = n^3$. Prove that $f(n)$ of this algorithm is in $O(n^3)$
9. What is the smallest value of n such that an algorithm whose running times is $100n^2$ runs faster than an algorithm whose running time is 2^n on the same machine?
10. Compare Big-Oh notation and Little-Oh notation.
11. Explain the best case and worst case of a search operation in a binary search tree.
12. Derive Average case running time of binary search.
13. Illustrate best case, average case and worst-case complexity with insertion sort algorithm.
14. Discuss the concept of best case, worst case and average case complexity of an algorithm with linear search algorithm.
15. What is the need of asymptotic analysis in calculating time complexity? What are the notations used for asymptotic analysis?
16. Calculate the time complexity for addition of two matrices.
17. Explain linear, quadratic, logarithmic and exponential time complexities. Write an algorithm that falls under each category.

18. Define the terms Best case, worst case and Average case time complexities.
19. Explain why the statement, “the running time algorithm A is at least $O(n^2)$ is meaningless
20. Why worst case analysis is given more importance than best case and average case analysis?
21. Define Big Oh, Big Omega and Theta notations and illustrate them graphically.
22. Show that for any real constants a and b where $b > 0$ $(n+a)^b = \Theta(n^b)$
23. We can extend our notation to the case of 2 parameters n and m that can go to infinity independently at different rates. For a given function $g(n,m)$ we denote by $O(g(n, m)) = \{f(n,m) : \text{there exist positive constants } c, n_0, \text{ and } m_0 \text{ such that } 0 \leq f(n,m) \leq cg(n,m) \text{ for all } n \geq n_0 \text{ and } m \geq m_0\}$. Give corresponding definitions for $\Omega(g(n,m))$ and $\Theta(g(n,m))$
24. Prove that $o(g(n)) \cap \omega(g(n))$ is an empty set
25. Find out the time complexity for the following algorithms

a. for i <-1 to n
 for j <-1 to n
 for k <-1 to n
 s++

b. for i <- 1 to n*n
 s++

c. A ()
 i = 1, s=1
 while (s<=n)
 i++
 s=s+i
 printf(“hi”)

d. A ()
 i =1
 for (i=1; i²<=n; i++)
 printf(“hi”)

e. A ()
 for (i=1; i <=n; i*2)
 printf(“hi”)

- f. A()
int i, j, k, n;
for (i=1; i<=n; i++)
 for(j=1; j<=i; j++)
 for(k=1; k<=100; k++)
 print("hi")
- g. A()
int i, j, k, n;
for (i=1; i<=n; i++)
 for(j=1; j<=i²; j++)
 for(k=1; k<=n/2; k++)
 print("hi")
- h. A()
int i, j, k;
for (i=n/2; i<=n; i++)
 for(j=1; j<=n/2; j++)
 for(k=1; k<=n; k=k*2)
 print("hi")
- i. A()
int i, j, k;
for (i=n/2; i<=n; i++)
 for(j=1; j<=n; j=2*j)
 for(k=1; k<=n; k=k*2)
 print("hi")
- j. Assume $n \geq 2$
A()
While($n > 1$)
 $n = n/2$
- k. A()
int i, j;
for (i=1; i<=n; i++)
 for(j=1; j<=n; j=j+i)
 print("hi")

26. Analyse the complexity of the following function

```
void function(int n)
{
    int count = 0;
    for (int i=n/2; i<=n; i++)
        for (int j=1; j<=n; j = 2 * j)
            for (int k=1; k<=n; k = k * 2)
                count++;
}
```

}

27. Analyze the complexity of the following program

```
main()
{
    for(int i=1; i<=n; i=i*2)
        sum = sum + I +func(i)
}

voidfunc(m)
{
    for(int j=1;j<=m;j++)
        Statement with O(1) complexity
}
```

28. Perform complexity analysis for the following code segments:

```
1) For i=1 to n do
    For j = 1 to n do
        A=B * C
    End for
End for
2) Function F(n)
{
    If(n==0)
        return(1)
    Else
        return (n*F(n-1))
}
```

29. Derive the recurrence equation for Fibonacci series. Perform complexity analysis for the same

30. Derive the recurrence equation for finding factorial of a number. Perform complexity analysis for the same

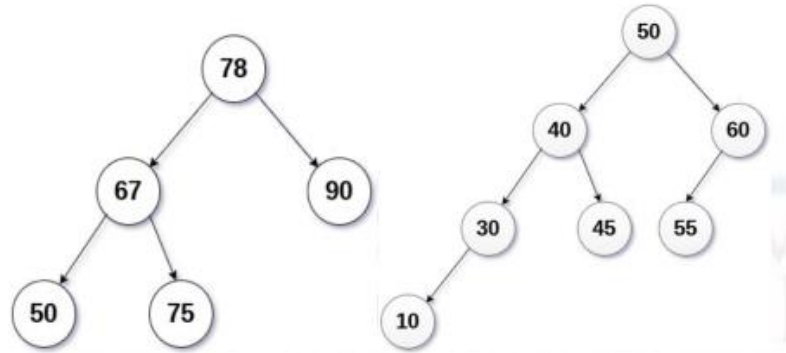
31. Apply Master's Theorem

- a. $T(n) = 7T(n/3) + n^2$
- b. $T(n) = 16T(n/4) + n^2$
- c. $T(n) = 7T(n/4) + n^2$
- d. $T(n) = 2T(n/4) + n$
- e. $T(n) = 2T(n/2) + n^2$
- f. $T(n) = 2T(n/2) + n$
- g. $T(n) = 2T(n/2) + n^3$
- h. $T(n) = T(9n/10) + n$
- i. $T(n) = 4T(n/2) + n^2\sqrt{n}$

- j. $T(n) = 8T(n/2) + n^2$
 - k. $T(n) = 4T(n/2) + n^2$
 - l. $T(n) = 4T(n/2) + n^3$
 - m. $T(n) = 2T(n/2) + n \log n$
 - n. $T(n) = 2^n T(n/2) + n^n$
 - o. $T(n) = 3T(n/2) + n^2$
 - p. $T(n) = 3T(n/3) + n/2$
 - q. $T(n) = 7T(n/2) + n^2$
 - r. $T(n) = 2T(n/4) + \sqrt{n}$
 - s. $T(n) = 3T(n/4) + n \log n$
 - t. $T(n) = 8T(n/2) + 100 n^2$
 - u. $T(n) = 2T(n/4) + 1$
32. Use a recursion tree to determine a good asymptotic upper bound on the recurrence
- a. $T(n) = 3T(n/2) + n$
 - b. $T(n) = T(n/2) + n^2$
 - c. $T(n) = T(9n/10) + 2T(n/10) + n$
 - d. $T(n) = T(n-1) + n$
 - e. $T(n) = T(xn) + T(1-x)n + n$
 - f. $T(n) = 2T(n/2) + n^2$
 - g. $T(n) = 2T(n/2) + n^2$
 - h. $T(n) = T(n/3) + T(2n/3) + n$
 - i. $T(n) = 3T(n/4) + n^2$
 - j. $T(n) = 4T(n/2) + n$
 - k. $T(n) = 2T(n/2) + n$
 - l. $T(n) = T(n/2) + 1$
 - m. $T(n) = 2T(n/2) + 1$
33. Argue that the solution to the recurrence $T(n) = T(9n/10) + T(n/10) + n$, where c is a constant is $\Omega(n \log n)$ by appealing to a recursion tree
34. Find the solution to the recurrence equation using iteration method:
- a. $T(2^k) = 3 T(2^{k-1}) + 1, \quad T(1) = 1$
 - b. $T(n) = 2T(n/2) + 2n + 3$
 - c. $T(n) = 2T(n/2) + n$
 - d. $T(n) = 3T(n-2)$

MODULE II

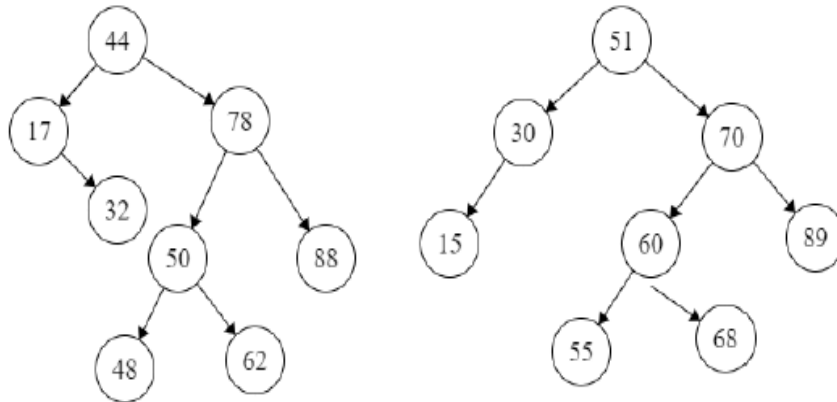
1. Explain AVL rotations with examples
2. Create an AVL tree by inserting the values from 1 to 8 in increasing order.
3. Insert the following keys into an initially empty AVL tree: 3,2,1,4,5,6,7
4. Perform the following operations in the given AVL trees.
 - i) Insert 70
 - ii) Delete 55



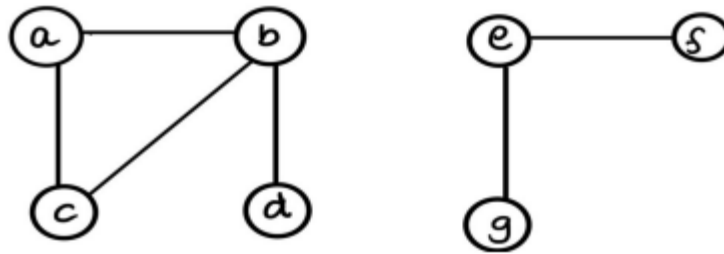
5. Define AVL tree. What is the advantage of AVL tree? Give an example.
6. Define AVL tree. Construct an AVL tree by inserting the keys: 44, 17, 32, 78, 50, 88, 48, 62, 54 into an initially empty tree. Write clearly the type of rotation performed at the time of each insertion.
7. Construct AVL tree by inserting following elements appeared in the order. 21, 26, 30, 9, 4, 14, 28, 18, 15
8. Perform the following operations in the given AVL trees.

1) Insert 54 in Tree 1

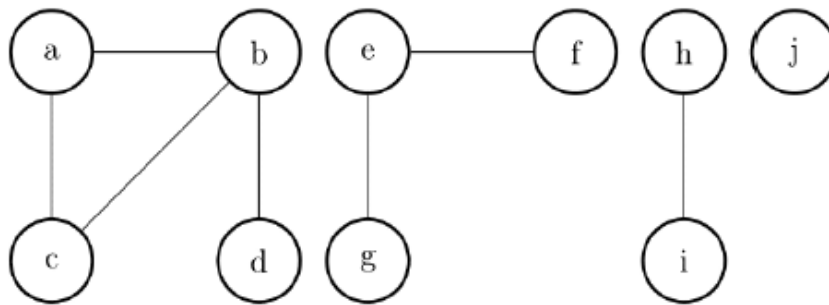
2) Delete 15 from Tree 2.



9. Give an algorithm for computing connected components in a graph using Disjoint Set data structure operations. Explain the working of this algorithm on the graph given below

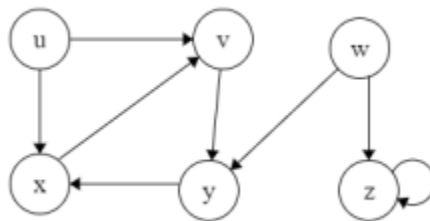


10. What are the operations supported by Disjoint Data Structure? Explain the working of Disjoint Set Data Structure for computing Connected Components of an undirected graph given in the following Figure.

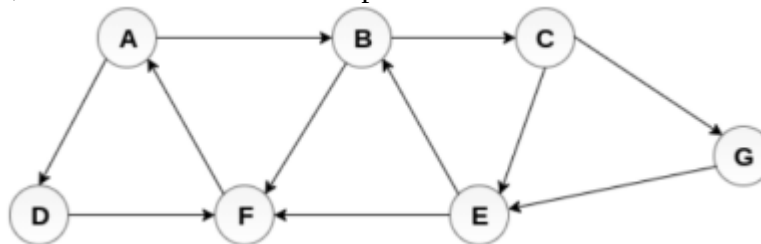


11. Define MAKE_SET(x), UNION(x,y) and FIND_SET(x) operations of disjoint set data structure with a suitable example.
12. State weighted rule (union by rank) and collapsing rule (path compression) applied in the disjoint set union and find operation respectively. How these rules will improve the efficiency of disjoint set operations.
13. Explain the UNION and FIND-SET operations in the linked-list representation of disjoint sets. Discuss the complexity.
14. Discuss briefly the heuristics, union by rank and path compression, to improve the running time of disjoint set data structure.
15. What are the operations possible on disjoint sets?
16. Show the UNION operation using linked list representation of disjoint sets
17. Give any one application of disjoint sets.
18. Explain the advantages of using height Balanced Trees? Explain AVL Rotations.
19. Find the minimum and maximum height of any AVL-tree with 7 nodes? Assume that the height of a tree with a single node is 0.
20. Explain how topological sorting can be performed using depth first search.

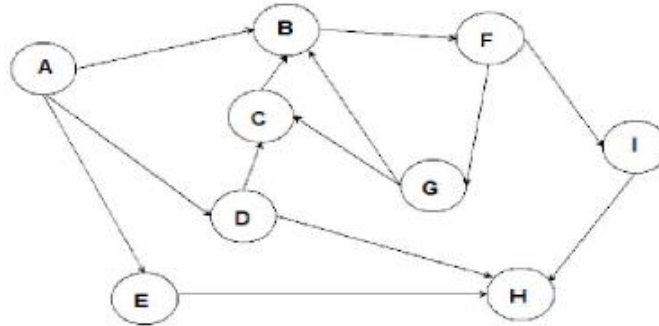
21. What are strongly connected components in a graph?
22. Write the applications of BFS and DFS
23. What are different classification of edges that can be encountered during DFS operation and how it is classified? Explain with example
24. Write an algorithm to perform BFS.
25. Write the algorithm for DFS and analyse its complexity.
26. Give Breadth First Search algorithm for graph traversal. Perform its complexity analysis.
27. Write a short note on graph traversals
28. Can we use DFS to detect cycles in a graph? Justify
29. Write an algorithm to print the path from a given source S to a vertex V using BFS.
30. Define topological sorting. Write a recursive algorithm to perform topological sorting
31. Define Strongly Connected Components of a graph. Write the algorithm to find Strongly Connected Components in a graph.
32. Perform DFS traversal on the following graph starting from node A. When multiple nodes are available for next traversal choose nodes in alphabetical order. Classify the edges of the graph into different category.



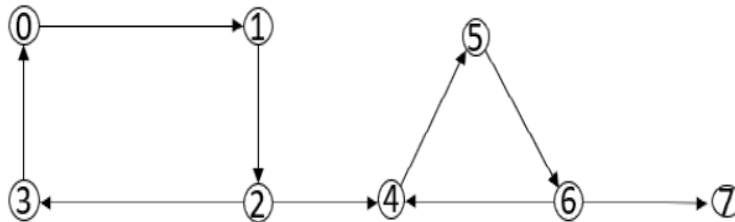
33. Write down BFS algorithm and analyse the time complexity. Perform BFS traversal on the given graph starting from node A. If multiple node choices are available for next travel, choose the next node in alphabetical order.



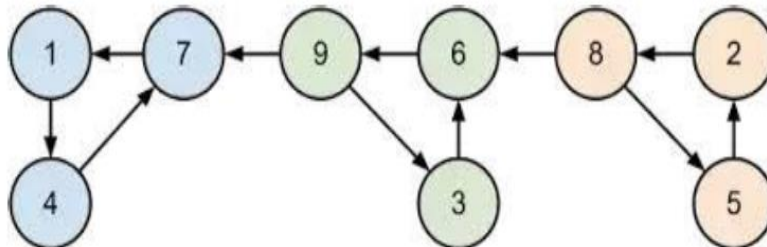
34. Perform DFS traversal on the above graph starting from node A. Where multiple node choices may be available for next travel, choose the next node in alphabetical order. Classify the edges of the graph into different category.



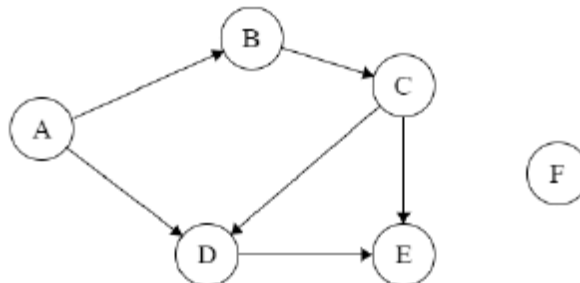
35. Find the strongly connected components of the given directed graph



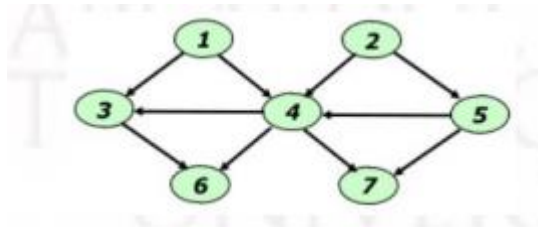
36. Find strongly connected components of the digraph using the algorithm showing each step



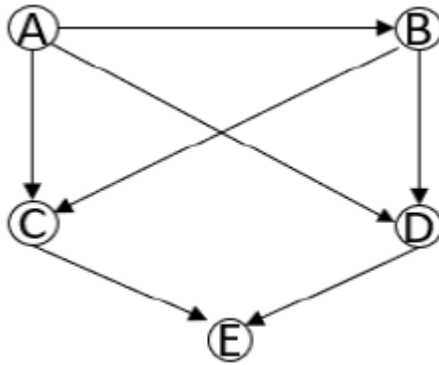
37. Find any ONE topological ordering of the following the graph



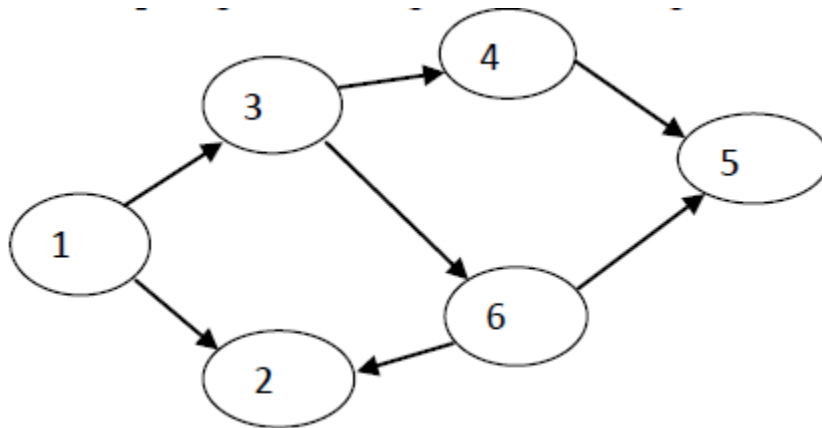
38. Find any three topological orderings of the given graph.



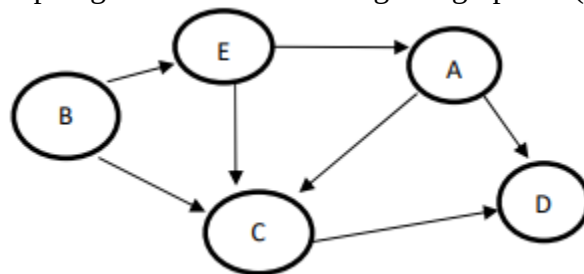
39. Find the different topological ordering of the given graph



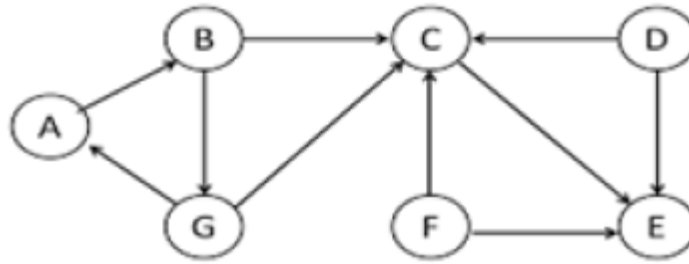
40. Write the topological sorting for the DAG given below



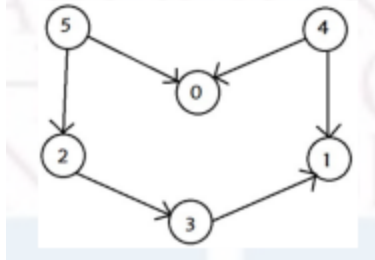
41. Obtain the topological sort order of the given graph $G=(V,E)$



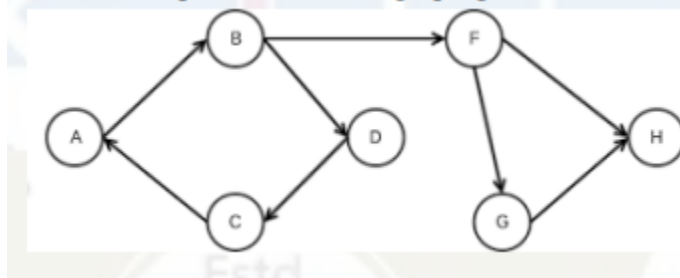
42. Perform BFS traversal on the above graph starting from node A



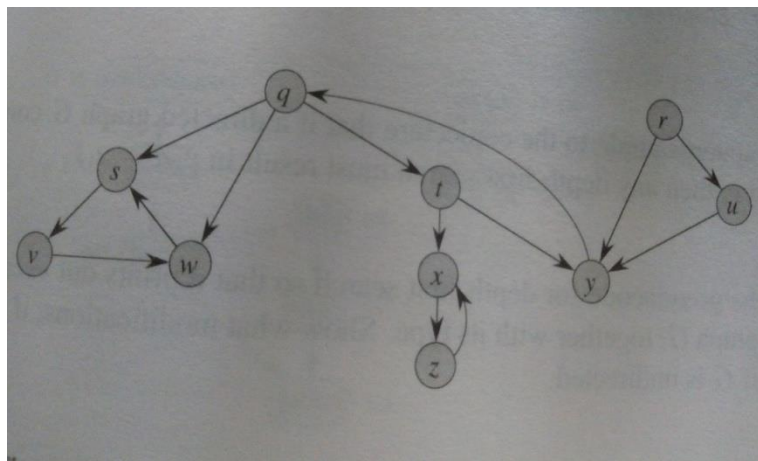
43. Find any two topological ordering of the DAG given below.



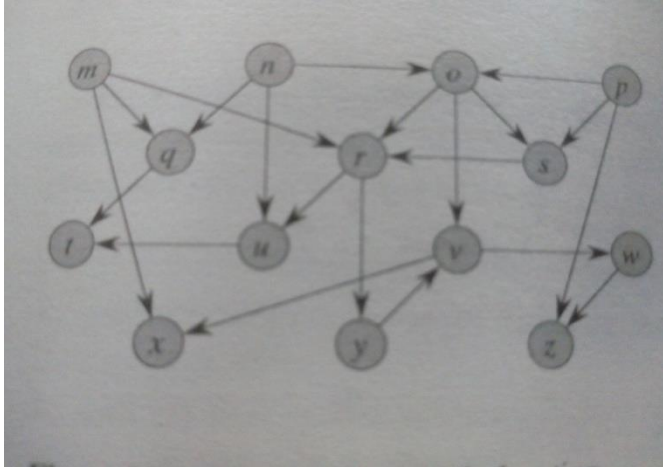
44. Find the strongly connected components of the digraph given below:



45. Show how the procedure Strongly connected components works on figure



46. Show the ordering of vertices produced by Topological Sort when it is run on the dag shown below



MODULE III

1. Write an algorithm based on divide-and-conquer strategy to search an element in a given list. Assume that the elements of list are in sorted order.
2. Why Strassen's matrix multiplication algorithm is better than traditional divide and conquer algorithm for multiplying two square matrices? What is the recurrence for the number of computational steps taken by Strassen's algorithm and its time complexity?
3. Write and explain merge sort algorithm using divide and conquer strategy. Also analyse the complexity.
4. Is Strassen's method of matrix multiplication suitable for practical applications? Justify your answer.
5. Explain Strassen's matrix multiplication and analyse its complexity
6. What is divide and conquer approach?
7. Explain 2-way merge sort with an example.
8. Why Strassen's matrix multiplication algorithm is better than traditional divide and conquer algorithm for multiplying two square matrices? What is the recurrence for the number of computational steps taken by Strassen's algorithm and its time complexity?
9. Multiply the following two matrices using Strassen's Matrix Multiplication Algorithm.

$$A = \begin{bmatrix} 6 & 8 \\ 9 & 7 \end{bmatrix} \quad B = \begin{bmatrix} 2 & 5 \\ 3 & 6 \end{bmatrix}$$

10. Using Strassen's multiplication method multiply the following matrices .

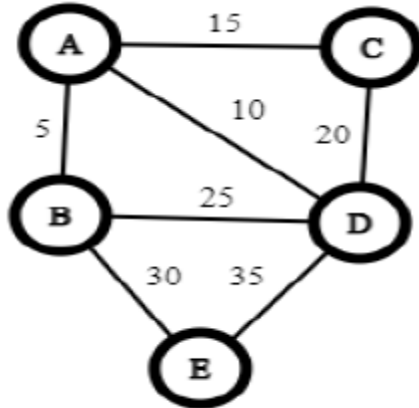
$$A = \begin{bmatrix} 1 & 5 \\ 3 & 8 \end{bmatrix} \quad B = \begin{bmatrix} 7 & 4 \\ 6 & 2 \end{bmatrix}$$

11. Illustrate the divide and conquer approach by applying 2 way merge sort for the input array: [15,12,14,17,11,13,12,16]. Write the recurrence for merge sort and give the complexity.
12. Write and explain merge sort algorithm using divide and conquer strategy using the data {30, 19, 35, 3, 9, 46, 10}. Also analyse the time complexity
13. Strassen's multiplication method is used to multiply two $n \times n$ matrices when n is a power of 2. How it can be modified when n is not a power of 2?
14. Give a control abstraction for greedy method.
15. Explain Greedy Approach. Write the general greedy algorithm.
16. Explain greedy strategy
17. Formulate Fractional Knapsack Problem. Write Greedy Algorithm for fractional Knapsack Problem.
18. How greedy method can be used to solve fractional knapsack problem. Explain with an example.
19. Consider an instance of fractional knapsack problem with 3 objects and capacity 20Kg. The profit and weight vectors corresponding to the 3 objects respectively are (10, 15, 20) and (8, 10, 20). Compute an optimal solution to this instance of fractional knapsack problem. What will be the profit earned corresponding to this solution?
20. Apply greedy algorithm for fractional knapsack to find the optimal ordering for loading the items in the knapsack. Let the knapsack capacity, $M=15$

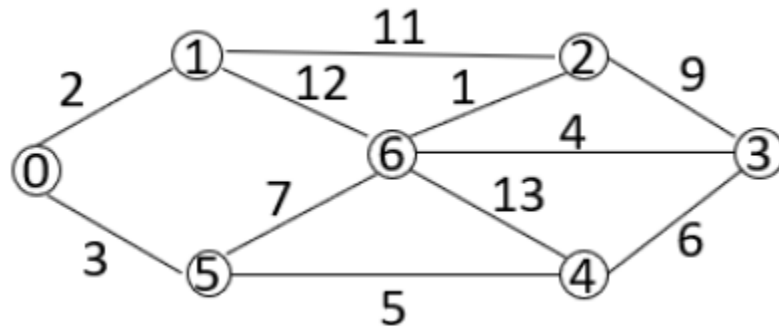
Items	Weight	Profit
1	8	24
2	9	18
3	5	20

21. Find the optimal solution for the following Fractional Knapsack problem. Given the number of items(n) = 7, capacity of sack(m) = 15, $W=\{1, 3, 5, 4, 1, 3, 2\}$ and $P = \{10, 15, 7, 8, 9, 4\}$
22. Consider the following instance of Fractional Knapsack problem with 3 objects. The capacity of the knapsack is 20 units. The weights and profits of the 3 items respectively are represented by the vectors $(w_1, w_2, w_3) = (18, 15, 10)$ and $(p_1, p_2, p_3) = (25, 24, 15)$. Using a greedy strategy compute the optimal solution to this instance.
23. Find the optimal solution for the following fractional Knapsack problem. Given number of items(n)=4, capacity of sack(m) = 60, $W=\{40, 10, 20, 24\}$ and $P=\{280, 100, 120, 120\}$
24. Find the optimal solution to a knapsack problem for the knapsack instance $n=8$ $m=110$, $(p_1, p_2, \dots, p_7) = (11, 21, 31, 33, 43, 53, 55, 65)$ and $(w_1, w_2, \dots, w_7) = (1, 11, 21, 33, 43, 53, 55, 65)$
25. Find an optimal solution to the fractional knapsack problem for an instance with number of items 7, Capacity of the sack $W=15$, profit associated with the items $(p_1, p_2, \dots, p_7) = (10, 5, 15, 7, 6, 18, 3)$ and weight associated with each item $(w_1, w_2, \dots, w_7) = (2, 3, 5, 7, 1, 4, 1)$
26. Find the optimal and all feasible solution for the knapsack problem. The sack maximum capacity is 100. The item weights and corresponding weights and profits are $W=\{10, 20, 30, 40, 50\}$ and $P = \{20, 30, 66, 40, 60\}$. Fill the sack such that the sack capacity is should not exceed the maximum capacity and objective of the problem is to maximize the profit
27. Write down Prim's algorithm and analyse the complexity

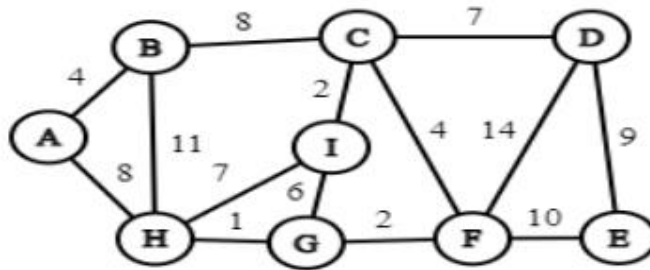
28. Write the Kruskal's algorithm for finding minimum cost spanning tree and explain the Kruskal's algorithm with an example. Analyse the complexity of the algorithm.
29. Can the minimum spanning tree of a graph be unique? Justify
30. Give Kruskal's algorithm for minimum cost spanning tree computation. Apply the algorithm to find the minimum cost spanning tree for the following graph



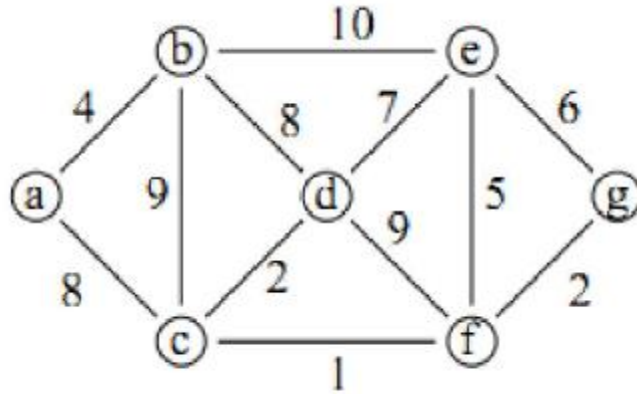
31. Apply Kruskal's algorithm for finding minimum cost spanning tree.



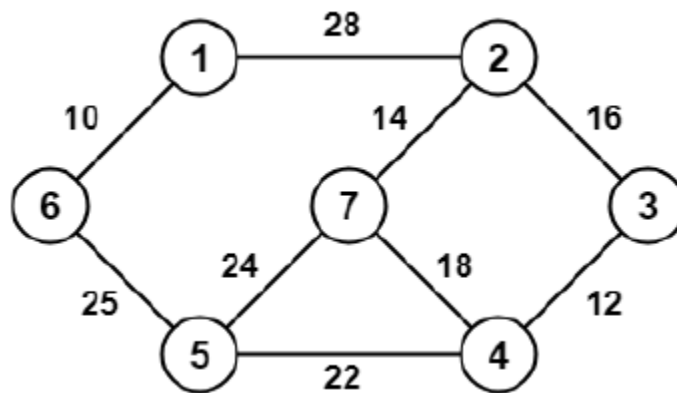
32. Apply Kruskal algorithm to find minimum cost spanning tree for the following graph



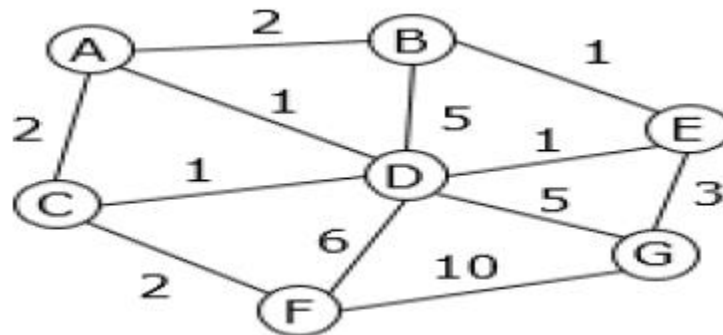
33. Apply Prim's algorithm on the graph given below.



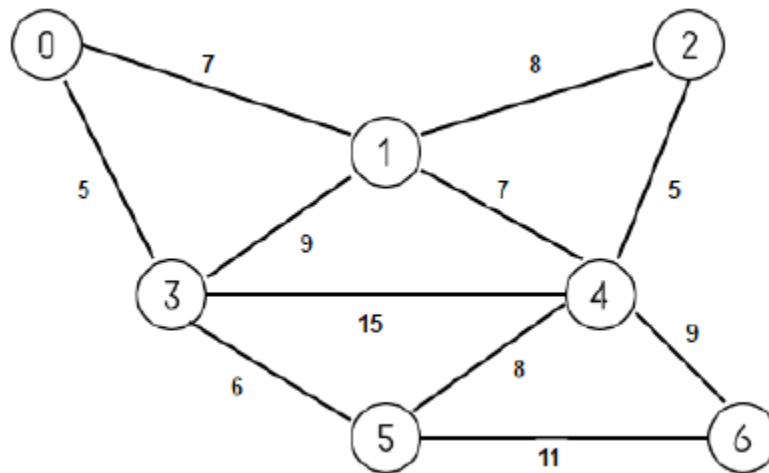
34. Construct the minimum spanning tree for the given graph using Kruskal's algorithm. Prove the correctness of Kruskal's algorithm.



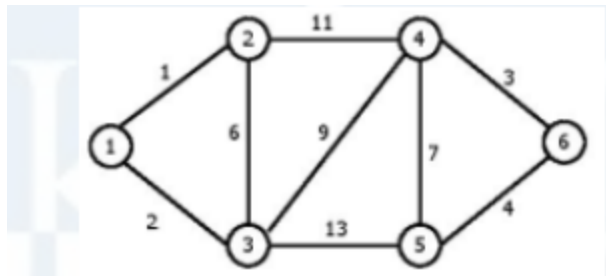
35. Apply Prim's algorithm on the following graph. Let A be the source vertex



36. Compute the Minimum Spanning Tree and its cost for the following graph using Kruskal's Algorithm. Indicate each step clearly.



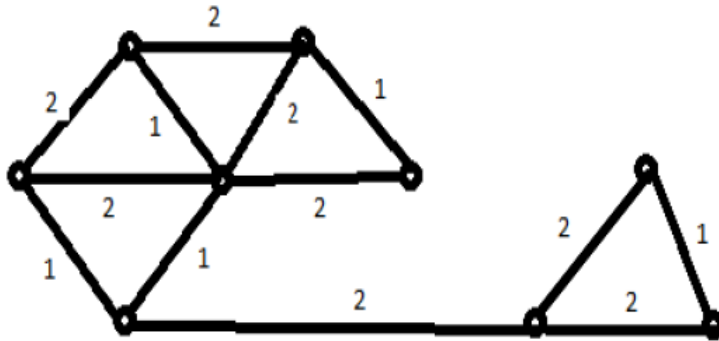
37. Construct the minimum spanning tree for the given graph using Kruskal's algorithm. Analyse the complexity of the algorithm.



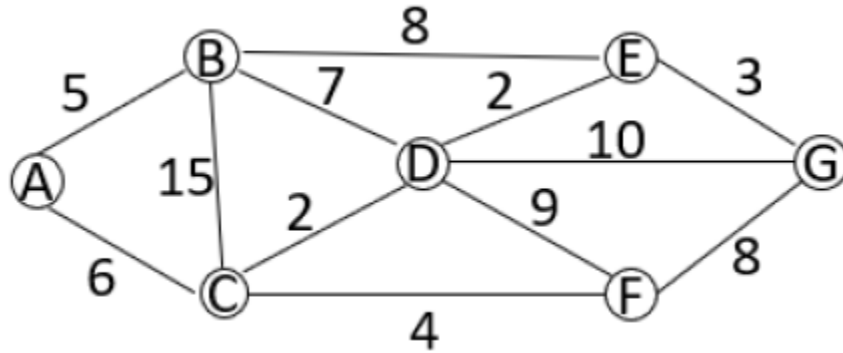
38. An undirected graph $G=(V, E)$ contains n ($n > 2$) nodes named $v_1, v_2, \dots, v_n$. Two vertices v_i, v_j are connected if and only if $0 < |i - j| \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$. What will be the cost of the minimum spanning tree (as a function of n) of such a graph with n nodes?
39. Consider a weighted complete graph G on the vertex set $\{v_1, v_2, \dots, v_n\}$ such that the weight of the edge (v_i, v_j) is $2|i-j|$. Find the weight of a minimum spanning tree of G .
40. Consider a complete undirected graph with vertex set $\{0, 1, 2, 3, 4\}$. Entry W_{ij} in the matrix W below is the weight of the edge $\{i, j\}$. What is the minimum possible weight of a spanning tree T in this graph such that vertex 0 is a leaf node in the tree T ?

$$W = \begin{pmatrix} 0 & 1 & 8 & 1 & 4 \\ 1 & 0 & 12 & 4 & 9 \\ 8 & 12 & 0 & 7 & 3 \\ 1 & 4 & 7 & 0 & 2 \\ 4 & 9 & 3 & 2 & 0 \end{pmatrix}$$

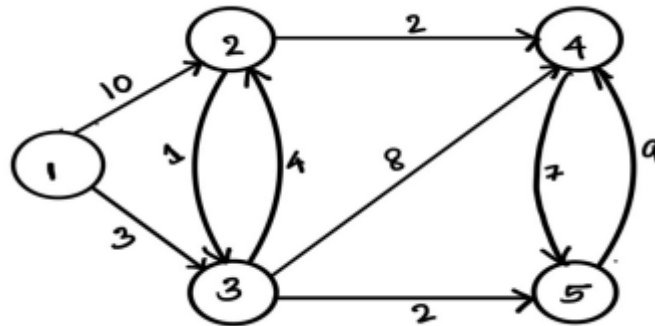
41. Let G be a weighted undirected graph with distinct positive edge weights. If every edge weight is increased by same value, will the minimum cost spanning tree and shortest path between any pair of vertices change. Justify your answer.
42. In a weighted graph, assume that the shortest path from a source 's' to a destination 't' is correctly calculated using a shortest path algorithm. Is the following statement true? If we increase weight of every edge by 1, the shortest path always remains same. Justify your answer with proper example.
43. Define spanning tree of a graph. Write the total number of spanning trees possible for a complete graph with 6 vertices.
44. Find the number of distinct minimum spanning trees for the weighted graph below



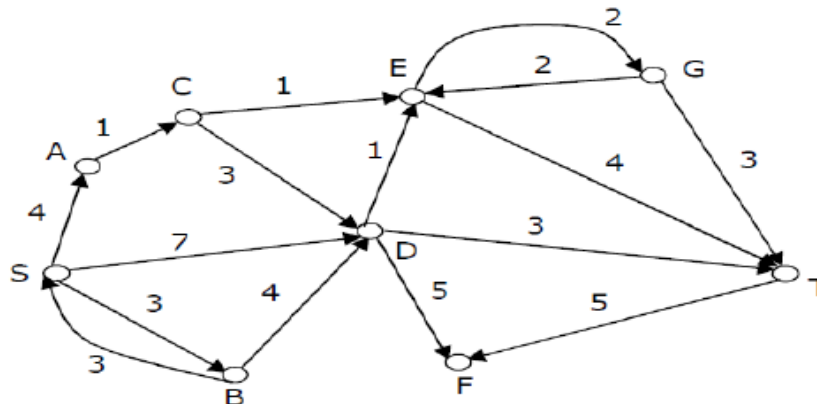
45. Apply Dijkstra's algorithm for finding the shortest path from vertex A to all other vertices



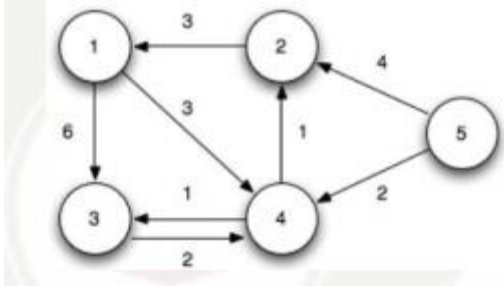
46. Apply Dijkstra's algorithm on the given graph to find the shortest path to all destinations starting from vertex with label 1



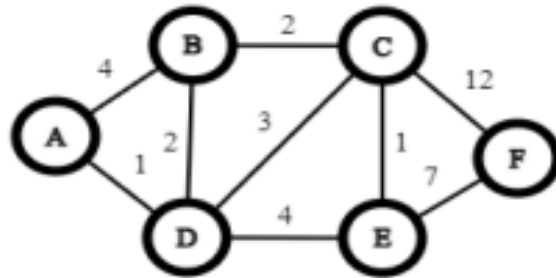
47. Write Dijkstra's Single Source Shortest path algorithm. Analyse the complexity. Find the shortest path from s to all other vertices in the following graph using Dijkstra's Algorithm.



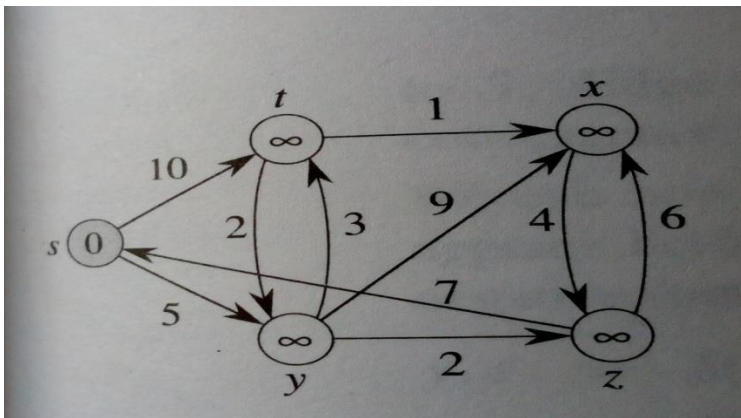
48. Write the pseudo code for Dijkstra's algorithm. Compute the shortest distance from vertex 1 to all other vertices using Dijkstra's algorithm.



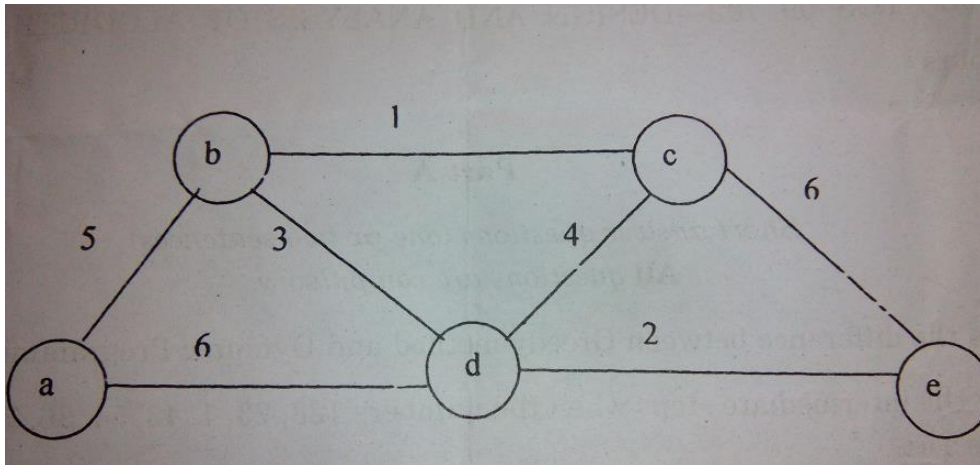
49. Apply Dijkstra's algorithm for single source shortest path to solve the following graph. Assume the source as node A.



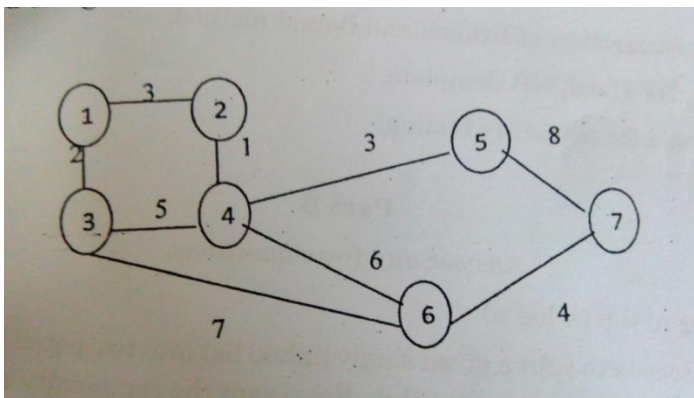
50. Find out the shortest path in the graph shown below using Dijkstra's algorithm



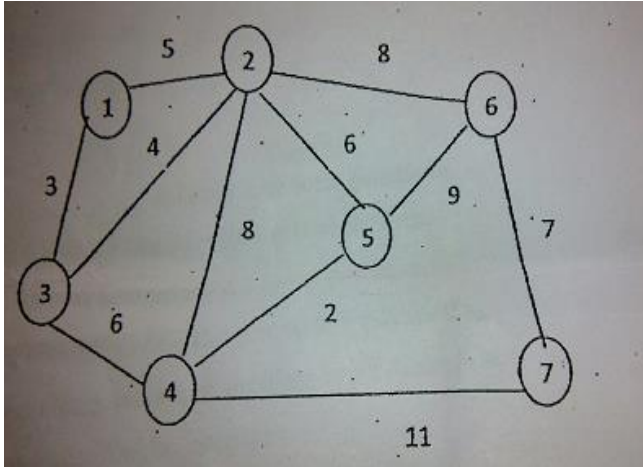
51. Write the Kruskal's algorithm apply it to find a minimum spanning tree for the following graph



52. Write Prim's algorithm and apply it to find the Minimum Spanning Tree for the following graph



53. Apply prim's algorithm and find minimum spanning tree for the following graph



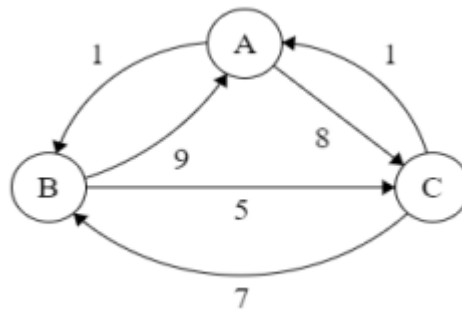
MODULE IV

1. State the principle of optimality.
2. Specify the difference between divide and conquer strategy and dynamic programming.
3. List and explain the characteristic properties associated with a problem that can be solved using dynamic programming.
4. Write and explain an algorithm to find the optimal parenthesization of matrix chain product whose sequence of dimension is given.
5. What are the characteristics of problems which can be solved by dynamic programming?
6. Explain about the structure of an optimal paranthesization of matrix-chain multiplication problem.
7. State Matrix Chain Multiplication Problem. Write Dynamic Programming Algorithm for Matrix Chain Multiplication Problem.
8. Using matrix chain multiplication as an example, illustrate the steps involved in finding solution of a dynamic programming problem
9. Discuss the elements of dynamic programming by considering the matrix chain multiplication problem.
10. Using Dynamic Programming, find the fully parenthesized matrix product for multiplying the chain of matrices $\langle A_1 A_2 A_3 A_4 A_5 A_6 \rangle$ whose dimensions are $\langle 30 \times 35 \rangle$, $\langle 35 \times 15 \rangle$, $\langle 15 \times 5 \rangle$, $\langle 5 \times 10 \rangle$, $\langle 10 \times 20 \rangle$ and $\langle 20 \times 25 \rangle$ respectively.

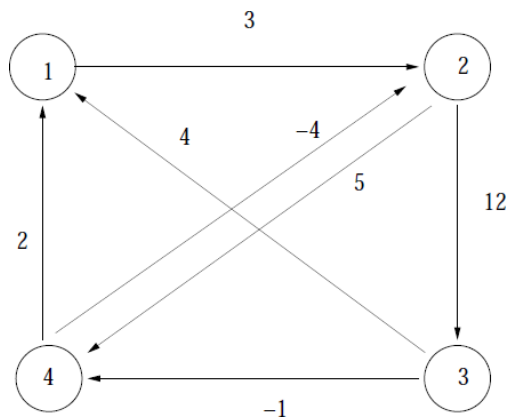
11. Given a chain of 4 matrices $\langle A_1, A_2, A_3, A_4 \rangle$ with dimensions $\langle 5 \times 4 \rangle, \langle 4 \times 6 \rangle, \langle 6 \times 2 \rangle, \langle 2 \times 7 \rangle$ respectively. Using Dynamic programming find the minimum number of scalar multiplications needed and also write the optimal multiplication order.
12. Consider the following four matrices and perform chain matrix multiplication using the dynamic programming approach. Finally give the optimal cost of multiplication and optimal parenthesization.

A	B	C	D
4x5	5x3	3x2	2x7

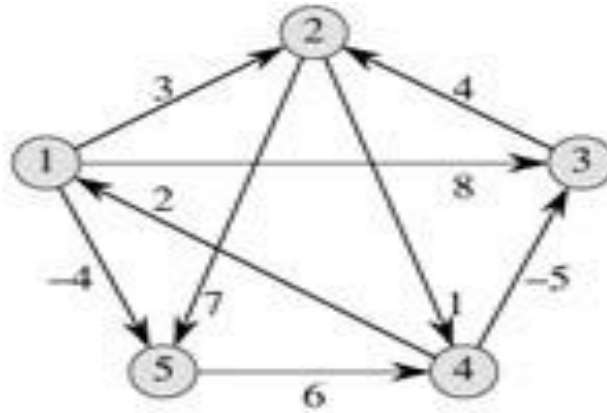
13. Write and explain the algorithm to find the optimal parenthesization of matrix chain product whose sequence of dimension is $4 \times 10, 10 \times 3, 3 \times 12, 12 \times 20$.
14. Discuss Floyd-Warshall algorithm for all pair shortest path problem. Solve the following instance using the algorithm.



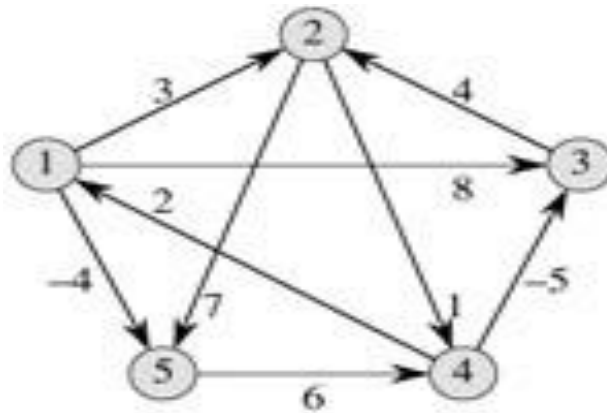
15. Write Floyd-Warshall's algorithm. We are given a weighted, directed graph $G = (V, E)$. Find the shortest path from the given graph using Floyd-Warshall's algorithm.



16. Construct the weight adjacency matrix of the given graph. Apply the FloydWarshall algorithm to construct the matrix D that represents the shortest paths distance between all vertices i and j ($1 \leq i \leq 5$ and $1 \leq j \leq 5$) through intermediate vertices 1 and 2.



17. Explain Floyd Warshall's algorithm and find the shortest path from each node to every other nodes in the given graph:

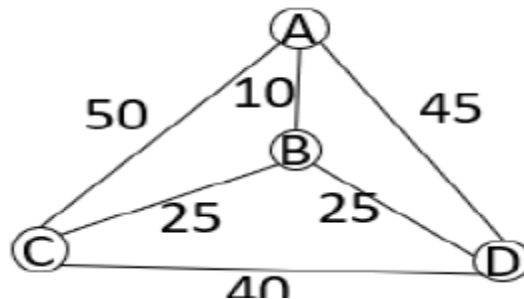


18. Discuss the control abstraction used in backtracking design technique. Draw the state space tree for 4-queens problem.
19. Explain the concept of Backtracking. Explain how 4 Queen problem can be solved using backtracking. Draw the state space tree corresponding to 4 Queen problem
20. Devise a backtracking algorithm to solve n-queens problem. Also find the time complexity of the algorithm.
21. Explain 0/1 knapsack problem. How backtracking can be used to solve it?
22. What is the difference between greedy and backtracking approaches to solving a knapsack problem? Explain the difference with the help of an example.

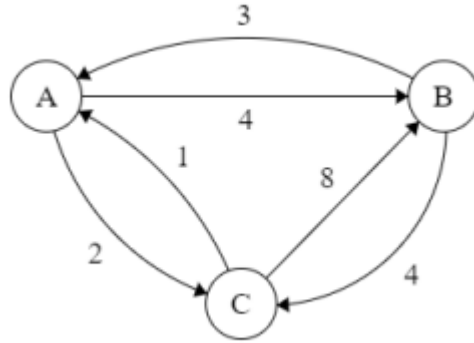
23. State and explain N-queens problem b) Draw the state space tree of 4-queens problem
24. Show the state space tree for 4 Queens problem. Show the steps in solving 4 Queens problem using backtracking method to print all the solutions
25. State and Explain N Queens Problem. Write the backtracking algorithm for solving N Queens problem.
26. Compare backtracking and branch-and-bound design techniques.
27. Define Travelling Salesman Problem (TSP). Explain the basic steps that are to be followed to solve TSP using branch and bound. Illustrate with an example.
28. Explain branch and bound technique.
29. Define Travelling Salesman Problem. Solve the following instance of TSP using branch and bound technique. The cost matrix is given below:

	A	B	C	D	E
A	-	10	8	9	7
B	10	-	10	5	6
C	8	10	-	8	9
D	9	5	8	-	6
E	7	6	9	6	-

30. Define TSP problem. Apply branch and bound algorithm for solving TSP.



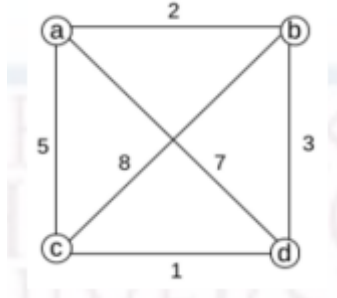
31. Define Travelling Salesman Problem (TSP). Apply branch and bound technique to solve the following instance of TSP. Assume that the starting vertex as A. Draw the state space tree for each step.



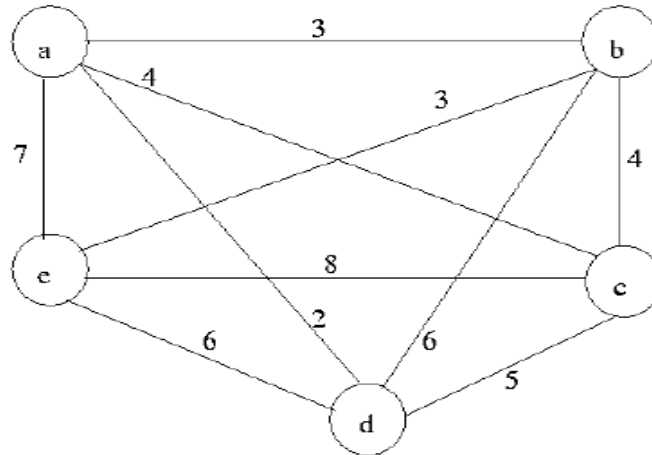
32. Find the minimum cost tour for the following Travelling salesman problem using Branch & Bound method

x	11	10	9	6
8	x	7	3	4
8	4	x	4	8
11	10	5	x	5
6	9	5	5	x

33. Define Travelling Salesman Problem (TSP). Apply branch and bound algorithm to solve TSP for the following graph, assuming the start city as 'a'. Draw the state space tree.



34. Solve Travelling Salesman problem for the following graph using Branch and Bound Technique.



MODULE V

1. Compare Las Vegas and Monte Carlo algorithms
2. Write short notes on Polynomial time reductions with example
3. Define class P and class NP
4. What do you mean by tractable problem? Give an example.
5. What are the steps to show that a given problem is NP-Complete?
6. State the difference between P, NP, NP-complete and NP-hard classes. Prove that travelling salesman problem is NP-complete.
7. Define NP hard and NP-Complete problems (b) Write short notes on Polynomial time reductions with example c) Define class P and class NP
8. Define the Travelling Salesman Problem and prove that it is NP-complete
9. Define NP- Hard and NP – Complete Problems. b) What are the steps used to show a given problem is NP-Complete? c) Write notes on polynomial time reducibility. Give Examples.
10. Prove that vertex cover problem is NP Complete. Discuss the approximation algorithm for vertex cover problem
11. State Cook's theorem. Prove that Hamiltonian cycle problem is NP Complete
12. Explain the clique problem and show that clique is NP complete
13. Write randomized quicksort algorithm and perform its expected running time analysis.
14. Discuss the need for approximation algorithm

15. Define approximation algorithm. Give an approximation algorithm for bin packing using first fit heuristic and give its approximation ratio.
16. Discuss the advantages of randomized algorithms over deterministic algorithms. Discuss Las Vegas and Monte Carlo algorithms with a suitable example.
17. State bin packing problem? Solve the bin packing problem for the given instance { .2, .5, .4, .7, .1, .3, .8} using next fit, first fit, best fit, worst fit and first fit decreasing
18. State bin packing problem? Solve the bin packing problem for the given instance { .4, .8, .1, .4, .2, .1} using next fit, first fit, best fit, worst fit and first fit decreasing. Bin capacity is 1.
19. State bin packing problem? Solve the bin packing problem for the given instance { .9, .8, .2, .2, .5, .4} using next fit, first fit, best fit, worst fit and first fit decreasing. Bin capacity is 1.
20. Specify the relevance of approximation algorithms
21. Define approximation algorithm. Give an approximation algorithm for bin packing using first fit heuristic and give its approximation ratio.
22. Define graph colouring problem.
23. Apply graph coloring approximation algorithm for the given graphs

